SESSION 1

Kiro Crew

우리 팀에 새로 들어온, 끄지 않는 팀원

Persistent | Self-Learning | Self-Evolving | Open Source | 발표 20분 | 상호 피드백 20분



오늘 이렇게 진행합니다

시간	구성	내용
0~2분	오프닝	오늘의 목표와 진행 방식
2~20분	세션 1	Kiro Crew — 무엇이고, 왜 다른가
20~40분	세션 2	Kiro CLI Tip 10개
40~55분	상호 피드백	각자 돌리는 것 3분 시연 → 옆사람 피드백
55~60분	랩업	다음까지 각자 해볼 것 1개 정하기

오늘의 목표는 하나입니다. 설치 안내가 아니라, **"내 업무 중 무엇을 넘길지"** 를 각자 하나씩 정해서 나가는 것.



문제는 "성능"이 아니었습니다

에이전트는 세션마다 훌륭하게 일합니다. 그런데 **매번 처음부터 다시 시작**합니다.

- 어제 정한 컨벤션을 오늘 다시 설명한다
- 같은 지적을 세 번째 하고 있다
- 터미널을 닫으면 그 맥락은 사라진다
- 내가 자리에 없으면 아무 일도 진행되지 않는다

> **핵심 진단**
>
> 문제는 모델의 능력이 아니라 **연속성의 부재**였습니다. 사람이 매번 컨텍스트를 다시 맞춰주는 것이 실제 비용의 대부분이었습니다.

Kiro Crew는 여기서 세 가지 아이디어로 출발했습니다 — **메모리**, **스스로 자라는 스킬**, **원격 실행 + 로컬 접근**.



한 문장 정의

> **"끄지 않고 계속 켜두는, 내 하드웨어에서 도는 오픈소스 AI 에이전트 워크스페이스"**

도구(tool)가 아니라 **팀원(teammate)** 에 가까운 존재입니다.

	일반 AI 어시스턴트	KIRO CREW
실행 시점	내가 말을 걸 때만	상시 구동
기억	창을 닫으면 리셋	메모리로 영속, 다음 세션이 이어받음
진입점	그 앱 하나	데스크톱 앱 · 대시보드 · CLI · Slack · Discord · Telegram · Teams 등
시작 조건	사람이 시작	스케줄 · 웹훅 · 하트비트로 스스로 시작
실행 위치	벤더 서버	**내 노트북 / 내 서버 / 내 계정의 EC2**

어디서 나왔나

Amazon 사내 사이드 프로젝트 `MeshClaw` 로 시작했습니다. 한 팀원이 주말에 자기 워크플로우를 실험한 것이 출발점이었습니다.

지표	값
사내 확산	6개월 이내 **39,000명 이상**의 빌더
기여자	약 **500명**
업데이트	**567회** / 주 평균 **143 커밋**
공개	2026년 8월, **Apache 2.0** 오픈소스
운영	공개 스티어링 위원회 · 공개 논의/의사결정

이 숫자가 중요한 이유

사내에서 39,000명이 먼저 검증한 뒤 오픈소스로 나온 드문 사례입니다. "만들어서 공개한 것"이 아니라 "이미 쓰이던 것을 공개한 것"입니다.



기존 Kiro 사용자라면 마이그레이션이 없습니다

Kiro Crew는 기존 `.kiro` 설정을 **그대로 읽습니다.**

- ✅ steering 파일
- ✅ Skills
- ✅ 커스텀 Agent
- ✅ MCP 서버 설정
- ✅ 모델 / 계정 설정 (kiro-cli의 것을 그대로 사용)

```
투자한 자산 → 그대로 동작
새로 배울 것 → 세션·메모리·스케줄·앱이라는 상위 개념뿐
```

포지셔닝

Crew는 Kiro를 **대체하지 않고 확장합니다.** IDE·CLI·Web이 "한 세션 안에서 내가 운전하는 것"이라면, Crew는 그 위에 **세션을 넘는 지속성**을 얹습니다.



구조 — 3계층만 알면 됩니다

1 사용자 인터페이스

데스크톱 앱 · 웹 대시보드 · CLI · Slack · Discord · Telegram · Teams

▼ 어느 입구로 들어와도 같은 에이전트

2 게이트웨이 (Gateway) — 상주 프로세스

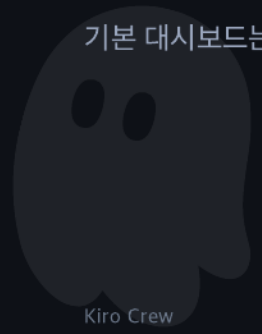
Python + aiohttp 단일 asyncio 프로세스 · 세션 연결 · 메모리 주입 · 승인 중개 · 안전장치 · 대시보드를 닫아도 계속 살아있음

▼ ACP (JSON-RPC 2.0 over stdio)

3 에이전트 백엔드 (kiro-cli)

LLM 호출 · MCP 도구 실행 · 모델 인증 관리

기본 대시보드는 `http://localhost:5476` (루프백 바인딩).



핵심은 "역할 분리"입니다

계층	하는 일	하지 않는 일
Gateway	세션 조율, 메모리·컨텍스트 주입, 스케줄 기동, 승인 중개, 정책 집행, 활동 노출	**추론하지 않음.** 모델을 직접 부르지 않음
kiro-cli	LLM 호출, MCP 도구 실행, 모델 인증	세션 관리·기억·스케줄을 하지 않음

이 경계가 만들어내는 결과

Gateway는 개별 대화보다 오래 사는 상주 계층입니다. 그래서 **대화가 끝난 뒤에도 기억이 남고, 정해진 시각에 스스로 일을 시작**할 수 있습니다.

세션은 Gateway가 재시작돼도 알아서 다시 붙습니다. 기본 30분 유휴로 닫혀도 메시지를 보내면 이어집니다.

세 가지 설계 원칙

Persistent — 지속성

세션·메모리·스케줄·태스크 체크포인트가 Gateway 재시작을 넘어 살아남습니다. 사람이 터미널에 없어도 예약·반응형 작업이 계속됩니다.

Self-Learning — 자기학습

교정과 실패가 **지속되는 교훈(lesson)** 이 됩니다.

> *"아니, 변경을 끝내기 전에 항상 프론트엔드 체크부터 돌려"* → 워크스페이스 범위의 교훈으로 저장되어 이후 세션에 적용

Self-Evolving — 자기진화

반복되는 패턴이 **재사용 가능한 스킬**로 합성됩니다. 메모리·교훈·스킬은 전부 Markdown이라 열어보고, 고치고, 지울 수 있습니다.

> **중요한 성질**: 세 가지 모두 **검사 가능(inspectable)** 합니다. 블랙박스로 학습하는 게 아니라, 무엇을 배웠는지 파일로 확인하고 편집합니다.



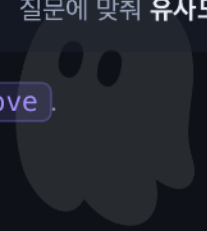
메모리 — 6종류, 2계층

종류	담는 것
선호 (Preferences)	사용자가 원하는 방식
프로젝트 (Projects)	진행 중 작업의 맥락
최근 히스토리	시간이 지나며 요약·감쇠
시맨틱 메모리	구조화된 사실
에피소드 메모리	과거 사건의 조각
교훈 (Lessons)	기본 동작을 덮어쓰는 교정 규칙

저장 방식이 둘로 갈립니다.

계층	위치	주입 방식
파일 기반	`~/.kiro/crew/workspace/memory/` (Markdown)	세션 시작 시 **통째로** 프롬프트에
DB 기반	`memory.db` (SQLite + FAISS 임베딩)	질문에 맞춰 **유사도 검색으로 선별**

메시지 30개마다 또는 유휴 시 자동 정리. *"이거 기억해"* 라고 하면 즉시 저장. CLI로는 `kirocrew learn add / list / remove`.



스스로 시작하는 다섯 가지 방식

모드	쓰는 곳	진입점
스케줄 (Cron)	브리핑, 감사, 정기 정비	`kirocrew cron` 또는 자연어로 요청
반응형 (Webhook)	CI 알림, 외부 자동화, 메시징 이벤트	인증된 에이전트 웹훅
하트비트	PR·배포·파이프라인 상태 변화 감시	조건 충족 시 작업 기동
태스크 러너	단계·테스트·검토가 명시된 유한 작업	`kirocrew run TASK.md`
서브에이전트	병렬 가능한 독립 워크스트림	`kirocrew spawn run "task"`

스케줄은 이렇게 만듭니다. — 채팅으로 *"매주 평일 오전 9시에 열린 작업을 요약해줘"*라고 말하면 잡이 만들어집니다. 표준 크론 표현식(`0 9 1-5`), 타임존, 휴일 스킵, 지터(동시 실행 분산)를 지원합니다.

> **안전장치**: 사람이 지켜보지 않는 실행이므로 **세션 단위 임시 신뢰(자동 승인)는 절대 붙지 않습니다.** 거부 목록·민감 경로 차단·거버넌스 정책은 그대로 적용됩니다. **비용 힌트**: 모델 호출 없이 스크립트/명령만 돌리는 잡은 **요청을 전혀 소모하지 않습니다.**



Apps — 채팅에 안 맞는 일은 화면으로

앱 하나가 **에이전트 · 스킬 · MCP 서버 · 크론 잡 · 대시보드 UI · 백엔드**를 골라 얻습니다. App Store에서 클릭 한 번으로 설치합니다.

앱	하는 일
Issue Radar	지정 저장소의 새 이슈·PR을 60초 주기로 폴링. Investigate(원인 조사) / Review(리뷰 초안) 버튼으로 세션 기동. 결론을 카드에 기록
Code Review Sage	변경 파일마다 별도 세션으로 리뷰, 영향 범위로 가중. PR 전체에 Red·Yellow·Green 판정. **GitHub에는 미제출(Draft) 상태로만 올림**
Spec Builder	막연한 아이디어 → 요구사항 → 설계 → 작업 목록. 단계마다 승인
Research Lab	질문을 좁은 하위 질문으로 쪼개 에이전트를 병렬 투입, 결과를 스트리밍
Agent Worlds	돌고 있는 에이전트를 픽셀아트 캐릭터로. 한눈에 바쁨 정도 파악

직접 만들 수도 있습니다. `app.json` 매니페스트에 얹을 것을 적으면 Crew가 읽어 연결합니다. TypeScript / Python App SDK 제공. 사내 전용 레지스트리를 붙이면 **내부 앱만 따로 배포**할 수 있습니다.



7계층 방어 — 오픈소스라서 검증 가능합니다

에이전트에게 실제 도구 접근 권한을 주므로, 통제는 **프롬프트가 아니라 런타임 경계에서** 집행됩니다.

계층	내용
1	**OS 샌드박스** — Linux namespace / macOS Seatbelt. standard·strict·off 모드
2	**도구 승인** — 대시보드나 Slack에서 검토 후 승인
3	**민감 경로 차단 · 쓰기 보호 경로**
4	**거부 명령 패턴** — **137개** 번들 패턴으로 파괴적 명령·유출 경로 차단
5	**의심스러운 bash 패턴 · MCP 입력 검증**
6	**출력 정제** — 자격증명 패턴 리댁션, URL 유출 탐지
7	**서명된 감사 로그 (SEL)** + 대시보드 토큰 인증

```
kirocrew security events   # 보안 이벤트 조회
kirocrew security audit    # 감사
kirocrew security verify   # 서명 검증
kirocrew policy show       # 거버넌스 천장 확인
```

거버넌스 천장: 정책은 **가장 엄격한 것이 이깁니다(tightest-wins).** 실행 중인 앱이나 에이전트는 허용 범위를 **좁힐 수는 있어도 엔터프라이즈 천장을 느슨하게 할 수는 없습니다.**
Windows는 fail-closed — OS 수준 샌드박스 등가물이 없으므로, 명시적 opt-in 없이는 에이전트 하위 프로세스를 아예 거부합니다.



멀티 인스턴스 — 노트북은 화면만, 일은 원격에서

여러 원격 Crew를 **하나의 대시보드에 탭으로** 붙여 씁니다.

- **노트북이 안 느려집니다** — 빌드·테스트·리포 스캔에 원격 박스의 CPU를 씁니다. "지금 노트북이 버틸까"가 아니라 "어느 박스에 맡길까"만 정합니다
- **내 노트북 상태와 분리됩니다** — 슬립에 들어가도, 네트워크가 바뀌어도 원격 작업은 계속됩니다. **30분짜리 마이그레이션을 걸어두고 회의에 들어갈 수 있습니다**
- **환경을 목적별로 쪼갭니다** — 리포마다 다른 툴체인. 망가지면 그 박스만 버립니다
- **데이터 옆에서 돌립니다** — VPC 안의 DB, 사내 전용 서비스. 터널을 오가는 건 대시보드 트래픽뿐이고 데이터는 원격에 남습니다
- **스펙을 필요한 만큼만 빌립니다** — 큰 작업 때만 `power` 티어로 올리고, 끝나면 `kirocrew cloud stop`

서울 리전 기준 월 비용 (기본 티어 t4g.xlarge, 4vCPU / 16GB)

실행 방식	COMPUTE	공인 IPV4	EBS 40GB	합계
24시간 상시 (730h)	~$121	~$3.65	~$3.65	**~$128/월**
08:00-01:00, 하루 17h	~$86	~$2.6	~$3.65	**~$92/월**
정지 (0h)	$0	$0	~$3.65	**~$4/월**

⚠️ 기본 스택은 기본 VPC를 씁니다. `--subnet`으로 프라이빗 서브넷을 지정하면 **NAT 게이트웨이(월 ~$43)가 별도로 붙고 인스턴스를 꺼도 계속 과금**됩니다. 개인 계정이라면 기본 경로가 훨씬 저렴합니다.



IDE · CLI · Web · Crew — 어떻게 다른가

제품	성격	잘 맞는 사람
IDE	코드 우선. 화면의 80%가 파일 트리 + 코드	직접 코드를 만지는 것을 선호
CLI	터미널 인터페이스, 헤드리스. CI/CD 실행에 유용	터미널에서 사는 사람
Web	원격 우선 샌드박스, 장시간 작업	로컬에 부담을 주고 싶지 않은 경우
Crew	세션을 넘는 지속성 · 병렬 오케스트레이션	여러 에이전트를 동시에 돌리는 사람

핵심 구분

IDE·CLI·Web은 **한 세션 안에서 내가 운전합니다.** Crew는 그 토대를 **세션 밖으로** 확장합니다 — 자체 메모리, 스케줄, 웹훅·CI 하트비트로 스스로 착수, 다중 에이전트 병렬 조율.

Crew는 kiro-cli 위에서 ACP로 돌고 기존 `.kiro` 설정을 읽습니다. **대체가 아니라 확장입니다.**



🆕 최신 업데이트 — Kiro Crew 0.9 (2026-10-16)



처음 자료를 모은 뒤(8/27)에 나온 릴리스입니다. 키워드는 **"크루메이트 상태를 한눈에"**입니다.

기능	무엇이 달라졌나	상태
Crewmate 프로필	크루메이트를 열면 목표 · 완료 · 진행 중 · 나를 기다리는 일 · 다음 한 걸음을 카드로 보여줌. 기록에서 읽으므로 **채팅 턴을 쓰지 않음**. Profile 카드에 권한 · 모델 · 추론 강도 · 스케줄	정식
크루메이트별 대시보드	프로젝트 리포트가 기본, 기본 페이지 12종. 채팅으로 스탠드업 · 로드맵 · 칸반을 요청. 숫자마다 출처 표시, **측정 안 한 비용은 0이 아니라 대시(-)**	Preview
원격 Crew를 라이브 창으로	다른 머신 Crew의 세션을 그쪽 transcript · 실행 상태 · 승인 그대로 창으로 엶. **이 머신에는 아무것도 저장하지 않음**. Online · Reconnecting · Error · Offline 배지	Preview
한도를 아는 컴포저	프롬프트가 모델 컨텍스트를 넘으면 경고, 넘친 전송은 한 번 더 보내야 나감. `! <명령>`으로 셸 실행, `Cmd+↑`로 마지막 메시지 수정	정식
사용량 예측	Usage 탭에 토큰 추정치, 지금 속도면 **크레딧이 언제 바닥나는지** 예측	정식 (토큰은 추정치)
채널	대시보드에서 연 세션을 Slack DM 스레드에 자동 연결(옵션), Telegram 포럼 토픽 하나 = 세션 하나	정식

> **14번과 이어집니다**
>
> 원격 Crew를 "탭으로 붙여 쓴다"던 것이, 0.9에서는 **그쪽 머신 화면을 그대로 여는 창**이 됐습니다. 데이터는 원격에 남는다는 원칙은 그대로입니다.

🆕 업그레이드 전에 알아둘 것 + 새 모델

0.9로 올리면 이렇게 바뀝니다

변화	영향
신규 설치의 답변 길이 기본값이 **Answer only**	기존 설정은 그대로 유지
Ultra 단계 폐지	Ultra로 저장된 설정은 Answer only로 동작
반투명 패널 기본 **Off**	Settings → Display에서 다시 켬
예전 relay 방식 채팅은 **읽기 전용**	열리지만 보낼 수 없음
메뉴 이름 · 위치 이동	Agent Capabilities → **Customize** / Subagent Settings · Warm Pool → **Settings → Agent Harness**

Crew에서 새로 쓸 수 있는 모델

모델	날짜	상태	포인트
Claude Haiku 5.5	10/23	Experimental	1M 컨텍스트. 100K 이하 **0.06x**, 초과 0.3x 크레딧
Claude Sonnet 5.5	10/21	**정식(GA)**	Experimental 라벨 해제. US · EU 지역
Kimi K3	10/15	Experimental	Moonshot AI 오픈 웨이트, 1M 컨텍스트, **2.4x** 크레딧

> 모델 목록 새로 고침은 `KIROCREW RESTART`
>
> 덤으로, IDE 1.4.0(10/19)에도 **세션을 넘는 Memory**가 들어왔습니다(1.4.10부터 기본 Off). 15번의 "IDE는 한 세션 안에서"라는 구분이 조금씩 흐려지고 있습니다.

이제 서로 보여줍시다 — 상호 피드백 20분

진행 방식

```
1) 시연  3분   — 지금 돌리고 있는 것 하나를 화면에 띄운다
2) 질문  1분   — 보는 사람이 궁금한 것만 묶어서
3) 피드백 1분  — 아래 세 가지 중 하나로만
```

피드백은 이 세 가지로만

유형	문장 틀
👍 **가져갈 것**	"그건 나도 당장 써보겠다 — 왜냐면 …"
🔧 **바꿀 것**	"그 단계는 스케줄/스킬로 넘길 수 있을 것 같다"
❓ **모르겠는 것**	"그게 왜 필요한지 아직 이해가 안 된다"

시연할 게 없다면, 이 셋 중 하나를 즉석에서

- **가장 오래 밀려 있는 이슈 하나**를 Issue Radar에 넣어 조사시켜 보기
- 반복해서 하는 지적 하나를 `kirocrew learn add`로 교훈으로 등록해 보기
- *"매주 평일 오전 9시에 열린 작업을 요약해줘"*를 스케줄로 만들어 보기

> **오늘 나갈 때 각자 하나씩 정합니다.**
>
> "다음까지 내가 Crew에 넘겨볼 업무 1개"



마무리

한 줄 요약

> **KIRO CREW는 지속성·자기학습·자기진화를 갖춘 에이전트 워크스페이스입니다.**
> 세션 단위 작업의 한계를 넘어 메모리·스케줄·앱·보안·감사를 내장하고, **내 하드웨어에서 돌며 Apache 2.0으로 공개**되어 함께 발전합니다.

잘 맡기는 요령

사람이 쥘 것	에이전트에 넘길 것
라벨을 실제로 적용하기	이슈 읽고 라벨 **제안**하기
리뷰를 공개하기 (Submit)	변경 읽고 리뷰 **초안** 쓰기
무엇을 만들지 결정하기	조사·정리·요약

결정을 위한 밑작업은 넘기고, 결정은 쥔다. 두 대표 앱이 실제로 그렇게 설계돼 있습니다.

처음부터 완벽한 자동화를 노릴 필요는 없습니다

하나만 골라 맡겨 보고, 결과가 믿을 만하면 스케줄과 앱을 하나씩 더합니다. 교훈이 쌓일수록 우리 팀의 컨벤션과 저장소를 아는 팀원에 가까워집니다.

> **지금 여러분의 저장소에서 가장 오래 밀려 있는 일 하나로 시작해 보세요.**

다음 세션 → Kiro CLI Tip 10개



🆕 부록 A · Crew 릴리스 한눈에 (0.1 → 0.9)

버전	날짜	한 줄 요약
0.1	8/4	**오픈소스 공개** — 10개 입구, 메모리, 스케줄, 137개 거부 패턴
0.2	8/9	브라우저 모드, 앱 8종(PPTX Maker 등), Windows, 한국어 · 일본어 UI
0.3	8/17	**Crew Mode**(병렬 스레드), 에이전트 전용 브라우저, 첫 턴 7.4초 → 0.35초
0.4	8/25	Windows · Linux 정식, **Secrets Vault**, 대시보드에서 코드 편집
0.5	8/29	**AWS Control** 앱, 함대 단위 보안 정책, 세션 탭, 세션 간 협업
0.6	9/5	**하네스 선택**(Claude Code · Codex · KAS), 원격 Crew 채팅, 4시간 턴
0.7	9/24	설정 즉시 반영, **재시작에도 살아남는 작업 큐**, PR 감시 확장
0.8	9/27	**조용한 감시**, 차단 사유 카드, Kiro CLI 내장, 에이전트 템플릿
0.9	10/16	크루메이트 프로필 · 대시보드 → 16 · 17번

두 달 반 동안 마이너 릴리스 9번

1~3주에 한 번꼴입니다. 오늘 본 내용도 한 달 뒤면 달라져 있을 수 있습니다.



🆕 부록 B · 8월 말 이후 달라진 것 — 네 갈래

0.5 ~ 0.8에서 생긴 변화를 방향별로 묶었습니다.

흐름	무엇이 생겼나	버전
오래, 끊기지 않고	채팅 턴 최대 4시간 · 서브에이전트 3시간. 작업은 내구성 큐에 들어가 게이트웨이를 재시작해도 복구. 업데이트는 진행 중인 작업이 끝날 때까지 기다림	0.6 · 0.7
필요할 때만 깨운다	PR 감시가 GitLab · Bitbucket Cloud · Azure DevOps까지. 리뷰어 질문 · 변경 요청에만 깨우고, 조용한 체크는 모델 턴을 쓰지 않음(Jev, Preview)	0.7 · 0.8
보안을 설명한다	보안 정책 하나를 URL로 배포해 여러 호스트에 적용. 가려진 자격증명 · 막힌 링크는 무엇이 왜 걸렸는지 카드로 보여줌	0.5 · 0.8
어떤 엔진으로든, 어디서든	하네스 선택(Claude Code · Codex · KAS · OpenCode · goose). 원격 Crew는 SSH · SSM · Fargate. 데스크톱 앱에 Kiro CLI 내장	0.6 ~ 0.8

세션 1의 메시지와 같은 방향입니다

"세션 밖으로 확장"이 더 **길게**(오래 도는 작업), 더 **조용하게**(필요할 때만 깨움) 가고 있습니다.



🆕 부록 C · 이미 쓰고 있다면 — 0.5 ~ 0.8 체크리스트

버전	확인할 것
0.5	받아쓰기 엔진이 로컬 하나로 통합 · S3 스냅샷 백업 폐지 → **AWS Control 앱**으로 · 일부 명령은 승인을 다시 물음
0.6	**Python 3.12 필수** · `agent.sandbox`를 끄지 말 것(자격증명 경로 보호가 샌드박스에 의존) · 무인 auto-run 계획은 기본 2시간 제한
0.7	`skills.max_triggered` 기본값 0 → 스킬은 자동 주입 대신 **검색해서 불러옴**
0.8	Crew log 기본 **On**(끄려면 `KIROCREW_CREW_LOG=0`) · command cron 검증 강화(중괄호 확장 등은 실패)
0.8 ⚠️	`mcp.json`의 **autoApprove 목록이 실제로 적용** → 승인 카드와 Crew 도구 게이트를 모두 건너뜀

가장 먼저 볼 한 줄은 마지막 줄입니다

예전에 적어 둔 autoApprove가 이제 진짜로 동작합니다. 게이트 뒤에 두려면 `mcp.honour_auto_approve`를 `false`로 두고 게이트웨이를 재시작합니다.

부록. 출처

소스	활용한 내용
kiro.dev/crew	제품 정의, 7계층 방어, 앱, 확산 지표, 라이선스·과금 FAQ
AWS 기술 블로그 — 우리팀에 새롭게 입사한 Kiro Crew	3계층 아키텍처, 메모리 6종·2계층, 스케줄, 앱, 멀티 인스턴스, 서울 리전 비용
github.com/kirodotdev/KiroCrew	설치 경로, 릴리스 채널, 137개 거부 패턴, 거버넌스 천장, 실행 모드
kiro.dev/changelog	16·17번 — Crew 0.9.0(10/16), 새 모델(10/15~10/23), IDE 1.4.0 Memory
kiro.dev/changelog (Crew)	22~24번 부록 — Crew 0.1.0(8/4) ~ 0.8.0(9/27) 릴리스 요약과 업그레이드 주의사항
🔒 비공개 세션 정리	18·19번 슬라이드 (Q&A 실제 질문, 비용 전략) — **사내 공유까지만**

